# Palmer Penguins — End-to-End ML Pipeline

Projekt końcowy warsztatów SGGW — Implementacja Systemów AI. Numer studenta # k035102@sggw.edu.pl

Klasyfikacja 3 gatunków pingwinów (Adelie, Chinstrap, Gentoo) na zbiorze **Palmer Penguins** (OpenML id: 42585) za pomocą kompletnego pipeline'u: DVC → Optuna → MLflow → BentoML.

---

## Zgodność z wymaganiami

### Zbiór danych

- **OpenML id: 42585** — 344 rekordy, po `dropna()` pozostaje 334
- Cechy numeryczne: `culmen_length_mm`, `culmen_depth_mm`, `flipper_length_mm`, `body_mass_g`
- Cechy kategoryczne: `island` (Biscoe/Dream/Torgersen), `sex` (MALE/FEMALE)

### Preprocessing (vs Titanic)

- Brakujące wartości: `dropna()` zamiast imputacji
- Kodowanie kategoryczne: `OneHotEncoder(sparse_output=False)` zapisany jako `models/encoder.pkl`
- Problem wieloklasowy: metryka `f1_score(average="weighted")`
- Encoder **współdzielony** między pipeline'em DVC a serwisem BentoML

### Pipeline DVC — 5 etapów

```
[download_data] → data/penguins.csv
[prepare_data]  → data/train.csv, data/test.csv, models/encoder.pkl
[train_model]   → models/model.pkl  (Optuna + MLflow, 20 prób)
[evaluate]      → metrics.json      (MLflow run "best-model")
[register_bentoml] → BentoML store
```

### Optuna + MLflow

- Eksperyment: `"penguins-optuna"`, ≥ 20 prób z `MLflowCallback`
- 4 hiperparametry: `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`
- Ewaluacja: `cross_val_score` 5-fold
- Run `"best-model"` z parametrami, metrykami i modelem z sygnaturą

### BentoML

- Dwa modele w store: `penguins_classifier` + `penguins_encoder`
- Serwis `PenguinsService` z endpointem `POST /predict`
- Wejście: 6 cech (4 numeryczne + island + sex)
- Wyjście: `{"species": "Adelie" | "Chinstrap" | "Gentoo"}`

---

## Struktura projektu

```
projekt-penguins/
├── params.yaml            # Parametry pipeline'u (DVC)
├── dvc.yaml               # Definicja 5 etapów pipeline'u
├── dvc.lock               # Fingerprints etapów (cache DVC)
├── metrics.json           # Metryki ostatniego uruchomienia
├── service.py             # Serwis BentoML /predict
├── bentofile.yaml         # Konfiguracja pakietu BentoML
```

```
├── requirements.txt       # Zależności projektu
├── src/
│   ├── download_data.py   # Etap 1: pobieranie z OpenML
│   ├── prepare_data.py    # Etap 2: preprocessing + encoder
│   ├── train_model.py     # Etap 3: Optuna + MLflow
│   ├── evaluate.py        # Etap 4: metryki + MLflow best-model
│   └── register_bentoml.py # Etap 5: rejestracja w BentoML store
├── data/
│   ├── penguins.csv       # Surowe dane (OpenML)
│   ├── train.csv          # Zbiór treningowy
│   └── test.csv           # Zbiór testowy
├── models/
│   ├── model.pkl          # Najlepszy RandomForestClassifier
│   └── encoder.pkl        # OneHotEncoder (island + sex)
└── screens/               # Screenshoty potwierdzające realizację
```

**params.yaml**

```yaml
data:
  openml_id: 42585
  test_size: 0.2
  random_state: 42
train:
  n_trials: 20
  cv_folds: 5
  experiment_name: "penguins-optuna"
```

---

## Uruchomienie

```bash
# 1. zależności
pip install -r requirements.txt

# cały pipeline
dvc repro

# sprawdzenie metryk
dvc metrics show

# 4. uruchom serwis predykcji
bentoml serve service:PenguinsService --port 3000
```

**Przykładowe zapytanie API**

```bash
curl -X POST http://localhost:3000/predict \
  -H "Content-Type: application/json" \
  -d '{"features": {"culmen_length_mm": 39.1, "culmen_depth_mm": 18.7,
       "flipper_length_mm": 181.0, "body_mass_g": 3750.0,
       "sex": "MALE", "island": "Torgersen"}}'
# → {"species": "Adelie"}
```

---

## Screenshoty

**Screenshot 1 — graf pipeline'u (dvc dag)**

```
C:\github\projekt-penguins>dvc dag
                    +---------------+
                    | download_data |
                    +---------------+
                            *
                            *
                            *
                     +--------------+
                     | prepare_data |
                  ***+--------------+****
            *****           *           *****
       *****                *                ****
   ****                     *                    *****
***                  +-------------+                  ***
  **                 | train_model |                 **
    **               +-------------+               **
      **             **            **            ***
        **         **                **        **
          **    **                    **    **
        +----------+            +------------------+
        | evaluate |            | register_bentoml |
        +----------+            +------------------+
```

dvc dag

Graf DAG pokazuje 5 etapów pipeline'u i ich zależności. Etapy wykonywane są sekwencyjnie: `download_data` → `prepare_data` → `train_model` → `evaluate` → `register_bentoml`. *DVC wykrywa zmiany i pomija etapy, których wejścia nie uległy zmianie (cache).

---

## Screenshot 2 — Wykonanie pipeline'u (dvc repro)

```
C:\github\projekt-penguins>dvc repro --force
Running stage 'download_data':
> python src/download_data.py
Pobieranie zbioru Palmer Penguins (OpenML id=42585)...
Pobrano 344 rekordow, kolumny: ['species', 'island', 'culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']
Dane zapisane do: data/penguins.csv

Running stage 'prepare_data':
> python src/prepare_data.py
Wczytywanie data/penguins.csv...
Przed dropna: 344 rekordow
Po dropna: 334 rekordow
Zbior treningowy: 267, testowy: 67
Zapisano data/train.csv i data/test.csv
Encoder zapisany do: models/encoder.pkl
Kategorie encodera: [array(['Biscoe', 'Dream', 'Torgersen'], dtype=object), array(['FEMALE', 'MALE', '_'], dtype=object)]

Running stage 'train_model':
> python src/train_model.py
Wczytywanie data/train.csv...
Zbior treningowy: 267 probek, 10 cech
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.schemas
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.tables
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.types
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.constraints
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.defaults
2026/03/09 15:40:09 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.comments
2026/03/09 15:40:10 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/03/09 15:40:10 INFO alembic.runtime.migration: Will assume non-transactional DDL.
C:\github\projekt-penguins\src\train_model.py:42: ExperimentalWarning: MLflowCallback is experimental (supported from v1.4.0). The interface can change in the future.
  mlflow_callback = MLflowCallback(
[I 2026-03-09 15:40:10,561] A new study created in memory with name: penguins-optuna
Uruchamianie 20 prob Optuna w eksperymencie 'penguins-optuna'...
[I 2026-03-09 15:40:10,548] Trial 0 finished with value: 0.9774283717679945 and parameters: {'n_estimators': 50, 'max_depth': 11, 'min_samples_split': 4, 'min_samples_leaf': 5}. Best is trial 0 with value: 0.9774283717679945.
[I 2026-03-09 15:40:11,955] Trial 1 finished with value: 0.9924528301886791 and parameters: {'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 6, 'min_samples_leaf': 4}. Best is trial 1 with value: 0.9924528301886791.
[I 2026-03-09 15:40:13,550] Trial 2 finished with value: 0.9849755415793151 and parameters: {'n_estimators': 300, 'max_depth': 15, 'min_samples_split': 6, 'min_samples_leaf': 4}. Best is trial 1 with value: 0.9924528301886791.
[I 2026-03-09 15:40:15,148] Trial 3 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 300, 'max_depth': 12, 'min_samples_split': 5, 'min_samples_leaf': 1}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:15,963] Trial 4 finished with value: 0.9849755415793151 and parameters: {'n_estimators': 150, 'max_depth': 14, 'min_samples_split': 8, 'min_samples_leaf': 4}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:16,554] Trial 5 finished with value: 0.9774283717679945 and parameters: {'n_estimators': 100, 'max_depth': 4, 'min_samples_split': 2, 'min_samples_leaf': 5}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:17,132] Trial 6 finished with value: 0.9849755415793151 and parameters: {'n_estimators': 100, 'max_depth': 4, 'min_samples_split': 9, 'min_samples_leaf': 1}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:17,465] Trial 7 finished with value: 0.9774283717679945 and parameters: {'n_estimators': 50, 'max_depth': 8, 'min_samples_split': 7, 'min_samples_leaf': 5}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:18,043] Trial 8 finished with value: 0.9924528301886791 and parameters: {'n_estimators': 100, 'max_depth': 7, 'min_samples_split': 4, 'min_samples_leaf': 4}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:18,308] Trial 9 finished with value: 0.9812019566736548 and parameters: {'n_estimators': 50, 'max_depth': 13, 'min_samples_split': 5, 'min_samples_leaf': 3}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:19,998] Trial 10 finished with value: 0.9886792452830189 and parameters: {'n_estimators': 300, 'max_depth': 11, 'min_samples_split': 10, 'min_samples_leaf': 1}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:21,985] Trial 11 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 200, 'max_depth': 10, 'min_samples_split': 6, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:22,418] Trial 12 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 250, 'max_depth': 11, 'min_samples_split': 2, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:23,496] Trial 13 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 200, 'max_depth': 10, 'min_samples_split': 4, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:24,049] Trial 14 finished with value: 0.9924528301886791 and parameters: {'n_estimators': 250, 'max_depth': 6, 'min_samples_split': 7, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:25,951] Trial 15 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 200, 'max_depth': 12, 'min_samples_split': 5, 'min_samples_leaf': 1}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:27,292] Trial 16 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 250, 'max_depth': 13, 'min_samples_split': 3, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:28,872] Trial 17 finished with value: 0.9924528301886791 and parameters: {'n_estimators': 300, 'max_depth': 9, 'min_samples_split': 6, 'min_samples_leaf': 3}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:29,728] Trial 18 finished with value: 0.9924528301886791 and parameters: {'n_estimators': 150, 'max_depth': 15, 'min_samples_split': 8, 'min_samples_leaf': 1}. Best is trial 3 with value: 0.9962264150943396.
[I 2026-03-09 15:40:30,813] Trial 19 finished with value: 0.9962264150943396 and parameters: {'n_estimators': 200, 'max_depth': 9, 'min_samples_split': 5, 'min_samples_leaf': 2}. Best is trial 3 with value: 0.9962264150943396.
Najlepsze hiperparametry: {'n_estimators': 300, 'max_depth': 12, 'min_samples_split': 5, 'min_samples_leaf': 1}
Najlepsza CV accuracy: 0.9962
Najlepszy model zapisany do: models/model.pkl
Updating lock file 'dvc.lock'

Running stage 'evaluate':
> python src/evaluate.py
Wczytywanie data/test.csv i models/model.pkl...
Accuracy: 1.0000
F1-score (weighted): 1.0000
Metryki zapisane do: metrics.json
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.schemas
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.tables
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.types
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.constraints
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.defaults
2026/03/09 15:40:37 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.comments
2026/03/09 15:40:37 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/03/09 15:40:37 INFO alembic.runtime.migration: Will assume non-transactional DDL.
2026/03/09 15:40:38 WARNING mlflow.models.model: 'artifact_path' is deprecated. Please use 'name' instead.
C:\Users\pucl\AppData\Local\Programs\Python\Python312\Lib\site-packages\mlflow\models\model.py:1209: FutureWarning: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats
 rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization.The recommended safe alternative is the 'skops' format.
  flavor.save_model(path=local_path, mlflow_model=mlflow_model, **kwargs)
Zalogowano run 'best-model' do MLflow.
Updating lock file 'dvc.lock'

Running stage 'register_bentoml':
> python src/register_bentoml.py
Ladowanie models/model.pkl i models/encoder.pkl...
Model zarejestrowany w BentoML: Model(tag="penguins_classifier:7iqr4qy3ymnpkaha")
Encoder zarejestrowany w BentoML: Model(tag="penguins_encoder:7mbq2py3ywkdqaha")
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.schemas
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.tables
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.types
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.constraints
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.defaults
2026/03/09 15:40:57 INFO alembic.runtime.plugins: setup plugin alembic.autogenerate.comments
2026/03/09 15:40:58 INFO alembic.runtime.migration: Context impl SQLiteImpl.
2026/03/09 15:40:58 INFO alembic.runtime.migration: Will assume non-transactional DDL.
Rejestracja zalogowana do MLflow.
Updating lock file 'dvc.lock'

To track the changes with git, run:

        git add dvc.lock

To enable auto staging, run:

        dvc config core.autostage true
Use `dvc push` to send your updates to remote storage.
```

dvc repro 1/2

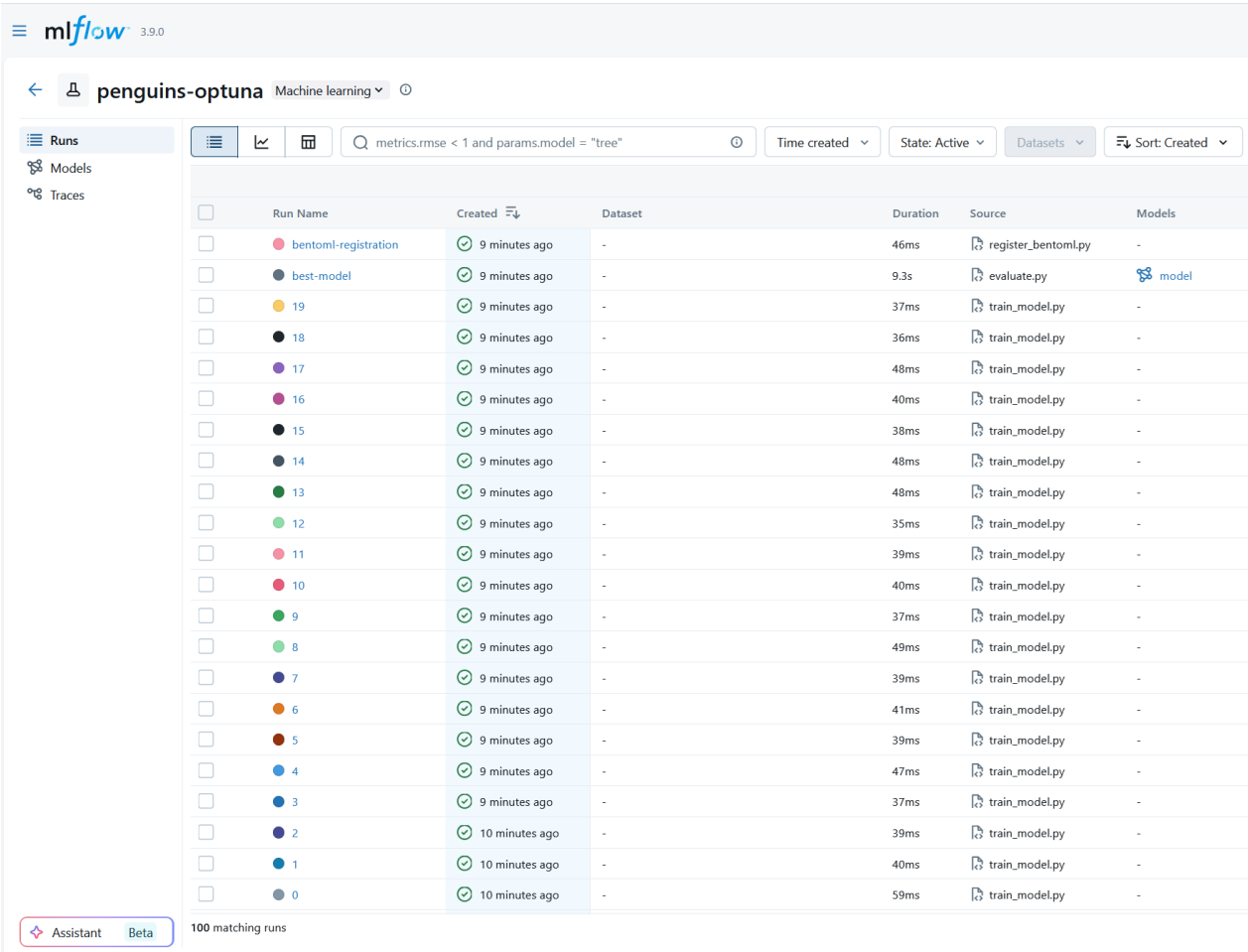
```
C:\github\projekt-penguins>dvc repro
Stage 'download_data' didn't change, skipping
Stage 'prepare_data' didn't change, skipping
Stage 'train_model' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Stage 'register_bentoml' didn't change, skipping
Data and pipelines are up to date.
```

dvc repro 2/2

`dvc repro` uruchamia kolejno wszystkie 5 etapów pipeline'u. Widoczne są logi poszczególnych etapów: pobieranie danych z OpenML, preprocessing (344→334 rekordów po `dropna()`), trenowanie z Optuna (20 prób), ewaluacja i rejestracja w BentoML store.

---

**Screenshot 3 — MLflow UI — lista runów eksperymentu**

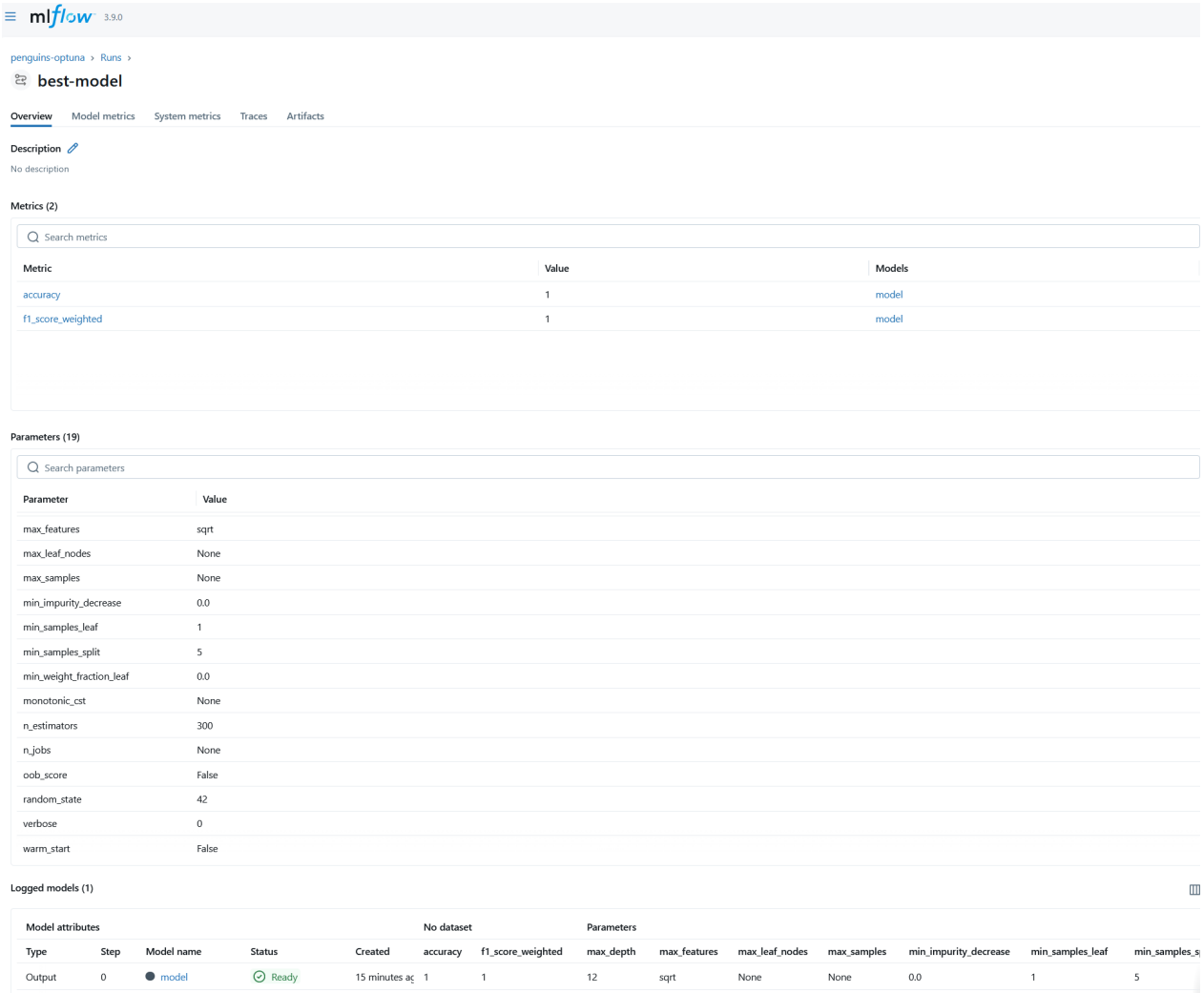



MLflow UI - lista runów

Interfejs MLflow pokazuje eksperyment `"penguins-optuna"` z ≥ 20 runami wygenerowanymi przez `MLflowCallback` Optuna. Każdy run odpowiada jednej próbie optymalizacji hiperparametrów i zawiera wartość metryki `cv_accuracy`.

---

**Screenshot 4 — MLflow UI — run “best-model”**

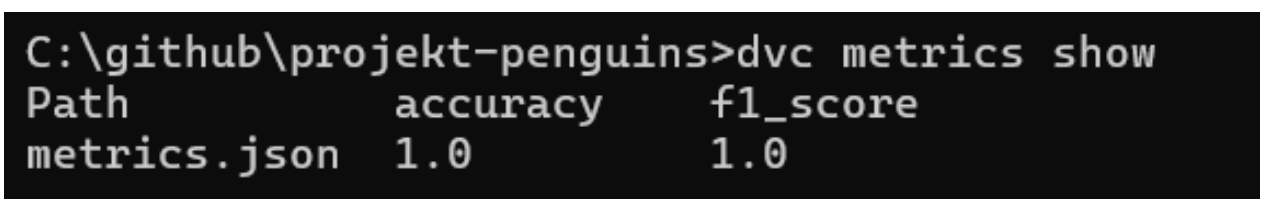



MLflow UI - best-model

Run "best-model" zawiera najlepsze hiperparametry znalezione przez Optuna (`n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`), metryki na zbiorze testowym (`accuracy`, `f1_score`) oraz zapisany artefakt modelu z sygnaturą (`mlflow.infer_signature`).

---

**Screenshot 5 — Metryki DVC (`dvc metrics show`)**



```
C:\github\projekt-penguins>dvc metrics show
Path          accuracy    f1_score
metrics.json  1.0         1.0
```

dvc metrics show

`dvc metrics show` wyświetla zawartość `metrics.json` śledzonego przez DVC. Metryki `accuracy` oraz `f1_score(average="weighted")` dla najlepszego modelu na zbiorze testowym.

---

**Screenshot 6 — Porównanie metryk (`dvc metrics diff`)**

```
C:\github\projekt-penguins>dvc metrics diff
Path          Metric    HEAD    workspace    Change
metrics.json  accuracy  1.0     0.9915       -0.0085
metrics.json  f1_score  1.0     0.9915       -0.0085
```

dvc metrics diff

`dvc metrics diff` porównuje metryki workspace z aktualnym commitem HEAD. Parametry zostały chwilowo pogorszone (`test_size: 0.35`, `n_trials: 3`), co spowodowało spadek accuracy — widoczna zmiana (Change) demonstruje działanie śledzenia metryk przez DVC.

---

**Screenshot 7 — Lista modeli BentoML (`bentoml models list`)**

```
C:\github\projekt-penguins>bentoml models list
 Tag                                Module           Size        Creation Time
 penguins_encoder:yobrg6q352f7waha  bentoml.sklearn  1.59 KiB    2026-03-09 20:32:52
 penguins_classifier:ykmaoma352wliaha  bentoml.sklearn  691.11 KiB  2026-03-09 20:32:50
 penguins_encoder:giod2yi352g7uaha  bentoml.sklearn  1.59 KiB    2026-03-09 20:28:48
 penguins_classifier:ge2wmrq352avqaha  bentoml.sklearn  529.73 KiB  2026-03-09 20:28:46
 penguins_encoder:7mbq2py3ywkdqaha  bentoml.sklearn  1.59 KiB    2026-03-09 15:40:55
 penguins_classifier:7iqr4qy3ywnpkaha  bentoml.sklearn  691.11 KiB  2026-03-09 15:40:54
 penguins_encoder:rxorznq3ykjjyaha  bentoml.sklearn  1.59 KiB    2026-03-09 15:16:24
 penguins_classifier:rtkdmia3ykvjsaha  bentoml.sklearn  524.75 KiB  2026-03-09 15:16:22
 penguins_encoder:o26pogi3yk262aha  bentoml.sklearn  1.59 KiB    2026-03-09 15:15:45
 penguins_classifier:owbw2qy3ykwoqaha  bentoml.sklearn  371.62 KiB  2026-03-09 15:15:43
 penguins_encoder:hm3ed6a3ykk3maha  bentoml.sklearn  1.59 KiB    2026-03-09 15:14:05
 penguins_classifier:hhzmwui3yki3eaha  bentoml.sklearn  504.64 KiB  2026-03-09 15:14:03
 penguins_encoder:37tri2y3ygxbcaha  bentoml.sklearn  1.59 KiB    2026-03-09 15:11:32
 penguins_classifier:33axd6y3yg32saha  bentoml.sklearn  110.42 KiB  2026-03-09 15:11:30
 penguins_encoder:pa2qifa3ygebsaha  bentoml.sklearn  1.59 KiB    2026-03-09 15:08:38
 penguins_classifier:o4rxy2i3ygnjmaha  bentoml.sklearn  524.75 KiB  2026-03-09 15:08:36
 titanic_logreg:3jyqs2i26sqmmaha    bentoml.sklearn  2.35 KiB    2026-03-08 14:43:56
 titanic_rf:3fxmeba26sf26aha        bentoml.sklearn  957.03 KiB  2026-03-08 14:43:54
 titanic_classifier:knifawa26kgi4aha  bentoml.sklearn  957.03 KiB  2026-03-08 14:25:50
 titanic_classifier:yjzvkya25wnfyaha  bentoml.sklearn  957.03 KiB  2026-03-08 13:53:09
```

bentoml models list

W BentoML store zarejestrowane są dwa modele: `penguins_classifier` (RandomForestClassifier) oraz `penguins_encoder` (OneHotEncoder). Enkoder jest współdzielony — zapisany w etapie `prepare_data` i używany zarówno w `evaluate`, jak i w serwisie `PenguinsService`.

---

## Screenshot 8 — Swagger UI / curl — endpoint `/predict`

None OAS 3.0

./docs.json

# penguins_classifier_service:dev

BentoML 1.4.35 | docs passing | Join BentoML Slack | Stars 8.5k | Follow BentoML

This is a Machine Learning Service created with BentoML.

## Help

- Documentation: Learn how to use BentoML.
- Community: Join the BentoML Slack community.
- GitHub Issues: Report bugs and feature requests.
- Tip: you can also customize this README.

Contact BentoML Team

Servers

.

## Service APIs

BentoML Service API endpoints for inference.

POST /predict

Parameters

Cancel | Reset

No parameters

Request body

application/json

```
{
  "features": {
    "culmen_length_mm": 39.1,
    "culmen_depth_mm": 18.7,
    "flipper_length_mm": 181.0,
    "body_mass_g": 3750.0,
    "sex": "MALE",
    "island": "Torgersen"
  }
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3000/predict' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
  "features": {
    "culmen_length_mm": 39.1,
    "culmen_depth_mm": 18.7,
    "flipper_length_mm": 181.0,
    "body_mass_g": 3750.0,
    "sex": "MALE",
    "island": "Torgersen"
  }
}'
```

Request URL

```
http://localhost:3000/predict
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |

```
{
  "species": "Adelie"
}
```

Download

Response headers

```
content-length: 21
content-type: application/json
date: Mon,09 Mar 2026 19:37:47 GMT
server: BentoML Service/penguins_classifier_service
x-bentoml-request-id: 37203c6ea353e58c
```

Responses

| Code | Description | Links |
|---|---|---|
| 200 | Successful Response | *No links* |

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
```

Serwis BentoML uruchomiony na porcie 3000 odpowiada na `POST /predict`. Wejście to 6 cech pingwina (4 numeryczne + `island` + `sex`), wyjście to `{"species": "Adelie" | "Chinstrap" | "Gentoo"}`. Preprocessing w serwisie jest identyczny jak w pipeline (ten sam `penguins_encoder` z BentoML store).